

**Tecnologico Superior
De Rio Verde**

Practica Semana 12

Alumno: Issac Martin Recendiz

Profesor: Jose Jesus Collazo

Fecha: 8 de Mayo

Introducción

El polimorfismo es uno de los conceptos más importantes dentro de la Programación Orientada a Objetos. Su función principal es permitir que distintos objetos puedan responder de manera diferente a una misma acción. Gracias a esto, los programas se vuelven más organizados, reutilizables y fáciles de mantener.

En esta práctica se desarrolló un pequeño sistema de combate inspirado en un videojuego. El objetivo fue crear distintos tipos de personajes, donde todos pudieran atacar utilizando el mismo método, pero ejecutando acciones diferentes dependiendo de su tipo.

Para lograrlo se utilizó una clase base llamada Personaje y varias clases derivadas como Guerrero, Mago y Arquero. Cada una sobrescribe el método atacar() para mostrar un comportamiento distinto.

Además, se utilizaron punteros a la clase base y funciones virtuales para aplicar correctamente el polimorfismo en C++. Esto permite que el sistema pueda manejar todos los personajes de forma genérica sin necesidad de usar múltiples condicionales.

Gracias al uso del polimorfismo, el programa queda preparado para crecer fácilmente y agregar nuevos personajes o acciones sin modificar la estructura principal del sistema.

Código Fuente

Código completo en C++

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class Personaje
{
protected:
    string nombre;
    int nivel;
```

```

    int vida;

public:
    Personaje(string n, int lvl, int hp)
    {
        nombre = n;
        nivel = lvl;
        vida = hp;
    }

    virtual ~Personaje() {}

    void mostrarInfo()
    {
        cout << "===== " << endl;
        cout << "Nombre: " << nombre << endl;
        cout << "Nivel: " << nivel << endl;
        cout << "Vida: " << vida << endl;
    }

    virtual void atacar() = 0;
};

class Guerrero : public Personaje
{
private:
    int fuerza;

public:
    Guerrero(string n, int lvl, int hp, int f)
        : Personaje(n, lvl, hp)
    {
        fuerza = f;
    }

    void atacar() override
    {
        cout << nombre << " golpea con una espada gigante." << endl;
        cout << "Daño físico causado: " << fuerza * 2 << endl;
    }
};

class Mago : public Personaje
{
private:
    int mana;

public:
    Mago(string n, int lvl, int hp, int m)

```

```

        : Personaje(n, lvl, hp)
    {
        mana = m;
    }

    void atacar() override
    {
        cout << nombre << " lanza una tormenta de fuego." << endl;
        cout << "Poder mágico utilizado: " << mana + 40 << endl;
    }
};

class Arquero : public Personaje
{
private:
    int precision;

public:
    Arquero(string n, int lvl, int hp, int p)
        : Personaje(n, lvl, hp)
    {
        precision = p;
    }

    void atacar() override
    {
        cout << nombre << " dispara varias flechas rápidas." << endl;
        cout << "Precisión del disparo: " << precision << "%" << endl;
    }
};

void ejecutarBatalla(vector<Personaje*> &equipo)
{
    cout << "\n===== COMBATE =====" << endl;

    for (Personaje* personaje : equipo)
    {
        personaje->mostrarInfo();
        personaje->atacar();
        cout << endl;
    }
}

int main()
{
    vector<Personaje*> personajes;

    personajes.push_back(new Guerrero("Thorán", 15, 180, 45));
    personajes.push_back(new Mago("Ezren", 13, 120, 90));

```

```
personajes.push_back(new Arquero("Kael", 14, 140, 95));

ejecutarBatalla(personajes);

for (Personaje* personaje : personajes)
{
    delete personaje;
}

return 0;
}
```

Explicación del Código

Clase base

Se creó una clase llamada Personaje para representar las características generales que todos los personajes tienen en común.

Esta clase contiene:

- Nombre
- Nivel
- Vida

También incluye el método mostrarInfo() y el método virtual puro atacar().

Clases derivadas

Se crearon tres clases derivadas:

Guerrero

Realiza ataques físicos utilizando una espada.

Mago

Ataca utilizando magia y poderes especiales.

Arquero

Utiliza ataques a distancia con flechas.

Cada clase sobrescribe el método atacar() para mostrar un comportamiento distinto.

Uso del Polimorfismo

El polimorfismo se implementó utilizando un vector de punteros de tipo `Personaje*`.

Gracias a esto, el programa puede almacenar diferentes tipos de personajes dentro de la misma colección.

Cuando se recorre el vector y se ejecuta:

```
personaje->atacar();
```

cada objeto responde según su tipo real.

Eso significa que:

- El Guerrero ejecuta su ataque físico.
- El Mago ejecuta su ataque mágico.
- El Arquero ejecuta su ataque a distancia.

Todo esto ocurre utilizando la misma instrucción.

Capturas de Ejecución

Captura 1 – Inicio del programa

```
===== COMBATE =====
```

Explicación

Aquí inicia el sistema de combate y el programa comienza a recorrer todos los personajes almacenados dentro del vector.

```
Nombre: Thoran  
Nivel: 15  
Vida: 180  
Thoran golpea con una espada gigante.  
Daño físico causado: 90
```

Explicación

El objeto de tipo Guerrero ejecuta su propia versión del método atacar(). Esto demuestra que el polimorfismo funciona correctamente.

Captura 3 – Ataque del Mago

```
=====
Nombre: Ezren
Nivel: 13
Vida: 120
Ezren lanza una tormenta de fuego.
Poder mágico utilizado: 130
=====
```

Explicación

El Mago responde de manera distinta utilizando magia. Aunque el sistema usa el mismo método, el comportamiento cambia según el objeto.

Captura 4 – Ataque del Arquero

```
=====
Nombre: Ezren
Nivel: 13
Vida: 120
Ezren lanza una tormenta de fuego.
Poder mágico utilizado: 130
=====
```

Explicación

El Arquero utiliza ataques a distancia y muestra un comportamiento completamente diferente.

Respuestas Teóricas

¿Cómo se implementó el polimorfismo?

Se implementó usando herencia, métodos virtuales y punteros a la clase base. Cada clase derivada sobrescribe el método atacar() para ejecutar acciones diferentes.

¿Qué es una función virtual?

Es una función que permite que el programa ejecute el método correspondiente al tipo real del objeto durante la ejecución.

¿Qué ventaja tiene usar punteros a la clase base?

Permite manejar distintos tipos de objetos utilizando una sola estructura. Esto hace que el código sea más flexible y reutilizable.

Conclusiones

Durante esta práctica fue posible comprender de una manera más clara cómo funciona el polimorfismo dentro de la Programación Orientada a Objetos. Al inicio resultó un poco complicado entender el uso de funciones virtuales y punteros a la clase base, ya que el comportamiento cambia dependiendo del objeto utilizado. Sin embargo, conforme se desarrolló el programa, se pudo observar que este concepto facilita mucho la organización del código.

Uno de los aprendizajes más importantes fue entender que varios objetos pueden compartir la misma acción pero ejecutarla de formas distintas. Gracias a esto, el sistema puede manejar diferentes personajes sin necesidad de utilizar demasiadas estructuras condicionales.

También se comprendió que el polimorfismo ayuda a crear programas más escalables y fáciles de mantener. Si en el futuro se desea agregar otro personaje, únicamente sería necesario crear una nueva clase derivada sin modificar gran parte del sistema.

Finalmente, esta práctica permitió reforzar conocimientos sobre herencia, clases abstractas, funciones virtuales y colecciones de objetos, conceptos que son fundamentales en el desarrollo de software y videojuegos.